R vs East & nQuery

Utilisation de R comme alternative aux logiciels propriétaires East et nQuery pour le calcul de Sample Size.

AROCENA Thomas

jeudi 8 janvier 2026

Résumé Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis sagittis posuere ligula sit amet lacinia. Duis dignissim pellentesque magna, rhoncus congue sapien finibus mollis. Ut eu sem laoreet, vehicula ipsum in, convallis erat. Vestibulum magna sem, blandit pulvinar augue sit amet, auctor malesuada sapien. Nullam faucibus leo eget eros hendrerit, non laoreet ipsum lacinia. Curabitur cursus diam elit, non tempus ante volutpat a. Quisque hendrerit blandit purus non fringilla. Integer sit amet elit viverra ante dapibus semper. Vestibulum viverra rutrum enim, at luctus enim posuere eu. Orci varius natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus.

Comparaison I : Sample size pour tester le hazard ratio dans un groupe à 2 bras.

Méthode de Schoenfeld

Le calcul de la taille d'échantillon de Schoenfeld, comme les autres méthodes de log rank se fait sous les hypothèses de loi exponentielles et de proportional hazard qui ne sont pas complètement satisfaisantes dans certain cas mais complètement ancré dans l'analyse de survie en générale.

Prenons α l'erreur de type I et β l'erreur de type II. Alors, en notant e le nombre d'evènement nécessaire, la formule (simple) de Schoenfeld donne :

$$e = 2 \times \frac{2\left(z_{1-\alpha/2} + z_{1-\beta}\right)^2}{\log(\mathrm{HR})} \tag{1}$$

Où z_x est le z-score de x, égale au quantile $(1-x)$ de la distribution normale standard.

Paramètres

On va comparer les resultats de Rpact et de East sur l'ensemble de test suivant :

$\alpha \in \{0.01, 0.05, 0.1, 0.20, 0.49\}$

$\text{power} \in \{0.51, 0.8, 0.9, 0.99\}$

$\text{HR} \in \{0.1, 0.5, 0.7, 0.9, 0.99\}$

$\text{Survie à 3 ans} \in \{0.1, 0.3, 0.6, 0.9\}$

```
param_list = list(
  alpha = c(0.01, 0.05, 0.1, 0.20, 0.49),
  power = c(0.51, 0.8, 0.9, 0.99),
  hr = c(0.1, 0.5, 0.7, 0.9, 0.99),
  surv_3y = c(0.1, 0.3, 0.6, 0.9)
)

param_table <-
  param_list |>
  expand.grid() |>
  as_tibble()
```

```
evaluate_relevancy_one <- function(
  alpha,
  power,
  hr,
  surv_3y
){
  if (alpha >= 0.4 || power <= 0.51 || power >= 0.99 || hr >= 0.99){
    relevancy <- "low"
  } else if (alpha >= 0.20 || hr <= 0.1 || hr >= 0.9 || surv_3y >= 0.9){
    relevancy <- "medium"
  } else {
    relevancy <- "high"
  }

  relevancy
}

evaluate_relevancy <- Vectorize(evaluate_relevancy_one)
```

This is a block with gray background and slightly rounded corners.

Rpact

On utilise la fonction `getSampleSizeSurvival` pour estimer le nombre d'évènements et de patients nécessaires.

```
sample_size_info <- getSampleSizeSurvival(
  alpha = 0.05, # Type I error
  sided = 2, # 2-sided Test
  beta = 0.1, # Type II error
  typeOfComputation = "Schoenfeld",
  hazardRatio = 0.7,
  pi2 = 0.3,
```

```
  followUpTime = 3,
  accrualTime = c(0, 3)
)

sample_size_info$eventsFixed
```

```
[1] 330.3779
```

```
sample_size_info$nFixed
```

```
[1] 3085.711
```

```
get_sample_size_wrapper <- function(
  alpha,
  power,
  hr,
  surv_3y,
  error = NA_real_
  ){
  tryCatch({
  # Times are in year NOT months
  sample_size_info <- getSampleSizeSurvival(
    alpha = alpha,
    beta = 1 - power,
    hazardRatio = hr,
    pi2 = 1 - surv_3y,
    eventTime = 3, # 3 years survival
    typeOfComputation = "Schoenfeld",
    sided = 2,
    allocationRatioPlanned = 1,
    dropoutRate1 = 0,
    dropoutRate2 = 0,
    followUpTime = 3, # 3 years of follow-up after accrual
    accrualTime = c(0, 3) # 3 years of accrual
  )

  return(tibble(
    e = ceiling(sample_size_info$eventsFixed),
    n = ceiling(sample_size_info$nFixed)))
  },
  error = function(er){
    return(tibble(e = error, n = error))
  })
}

#get_sample_size_wrapper(0.49,0.51,0.1,0.6)
```

```
p_res <-
  param_table |>
```

```r
  mutate(
    nested_res = pmap(param_table,get_sample_size_wrapper)
    ) |>
  unnest(nested_res)

#head(p_res) ▷ gt()
#write_csv(p_res, "results/rpact_grid.csv")
#write_csv2(p_res, "results/rpact_grid.csv")
```

East

```r
# As East has problems running the 400 inputs consecutively
# I ran 4*100 by spliting by power value
east_res_dirty <- bind_rows(
  read_csv("results/east_pwr051.csv"),
  read_csv("results/east_pwr08.csv"),
  read_csv("results/east_pwr09.csv"),
  read_csv("results/east_pwr099.csv")
)
```

```
New names:
Rows: 76 Columns: 24
── Column specification ────────────────────────────────────────────────────────
Delimiter: "," chr
(4): ID, Design Type, Test Type, Var (Log HR) dbl (19): Sr. No., No. of
Looks,
Specified α, Power, nt/nc, Sample Size, Exp ... lgl (1): ... 24
ℹ Use `spec()` to retrieve the full column specification for this data. ℹ
Specify the column types or set `show_col_types = FALSE` to quiet this
message.
New names:
Rows: 96 Columns: 24
── Column specification ────────────────────────────────────────────────────────
Delimiter: "," chr
(4): ID, Design Type, Test Type, Var (Log HR) dbl (19): Sr. No., No. of
Looks,
Specified α, Power, nt/nc, Sample Size, Exp ... lgl (1): ... 24
ℹ Use `spec()` to retrieve the full column specification for this data. ℹ
Specify the column types or set `show_col_types = FALSE` to quiet this
message.
New names:
Rows: 96 Columns: 24
── Column specification ────────────────────────────────────────────────────────
Delimiter: "," chr
(4): ID, Design Type, Test Type, Var (Log HR) dbl (19): Sr. No., No. of
Looks,
Specified α, Power, nt/nc, Sample Size, Exp ... lgl (1): ... 24
ℹ Use `spec()` to retrieve the full column specification for this data. ℹ
Specify the column types or set `show_col_types = FALSE` to quiet this
message.
```

```
New names:
Rows: 100 Columns: 24
── Column specification ────────────────────────────────────────────────────────
Delimiter: "," chr (4): ID, Design Type, Test Type, Var (Log HR) dbl (19): Sr. No., No. of Looks,
Specified α, Power, nt/nc, Sample Size, Exp … lgl (1): …24
ℹ Use `spec()` to retrieve the full column specification for this data. ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
• `` -> `…24`
```

```r
# Keep only needed input/output
# and add the survival rate at 3 years that is
# missing in East output
east_res <- east_res_dirty |>
  select(
    alpha = "Specified α",
    power = "Power",
    hr = "Hazard Ratio (Alt.)",
    e = "Maximum Events",
    n = "Sample Size"
  ) |>
    arrange(alpha, power, hr, n) |>
    mutate(surv_3y = rep(c(0.1, 0.3, 0.6, 0.9), n()/4)) |>
    mutate(power = signif(power, 1))
```

nQuery

```r
nquery_res_dirty <- read.csv2("results/nquery_grid.csv", dec = ",")

nquery_res_less_dirty <- data.frame(t(nquery_res_dirty[-1]))
colnames(nquery_res_less_dirty) <- nquery_res_dirty[,1]

nquery_res <-
  nquery_res_less_dirty |>
  tibble() |>
  select(
    alpha = "Test Significance Level, ??",
    power = "Power (%)?",
    hr = "Hazard Ratio, h=??/???",
    lambda1 = "Group 1 Exponential Parameter, ???",
    e = "Total Number of Events Required, E?",
    n = "Sample Size per Group, n?"
  ) |>
    mutate(
      across(
        c(alpha, power, hr, lambda1),
        \(x){parse_number(x, locale = locale(decimal_mark = ","))}
      ),
      lambda1 = signif(lambda1, 1),
      surv_3y = case_when(
```

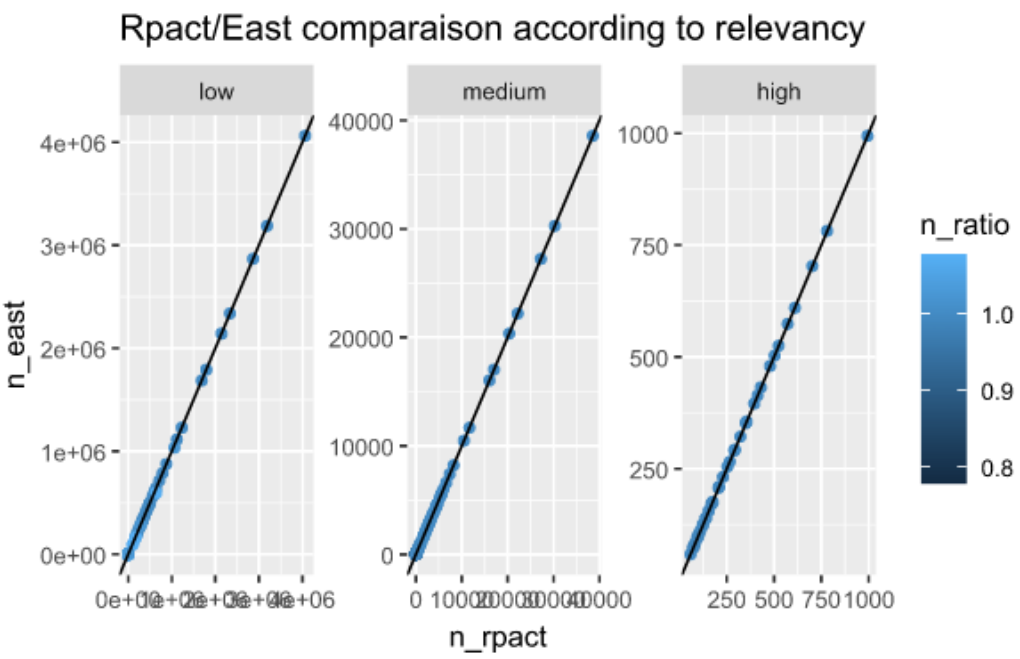
```
        lambda1 == 0.80 ~ 0.1,
        lambda1 == 0.40 ~ 0.3,
        lambda1 == 0.20 ~ 0.6,
        lambda1 == 0.04 ~ 0.9,
        .default = NA_real_
      ),
      across(c(e, n), as.numeric),
      power = power/100,
      n = 2*n # introduce an error cause all n will be even.
    ) |>
    select(- lambda1)
```

rpact vs East

```
combined_res <-
  p_res |>
  inner_join(
    east_res,
    by = join_by(alpha, power, hr,surv_3y),
    suffix = c("_rpact", "_east")
    ) |>
  mutate(
    e_ratio = e_rpact/e_east,
    n_ratio = n_rpact/n_east
  )
```

```
combined_res <-
  combined_res |>
    mutate(relevancy = evaluate_relevancy(alpha, power, hr, surv_3y)) |>
    mutate(relevancy = fct_relevel(relevancy,c("low", "medium", "high")))
```

```
ggplot(combined_res) +
  aes(x = n_rpact, y = n_east, color = n_ratio) +
  geom_point() +
  geom_abline() +
  facet_wrap(~relevancy, scales = "free") +
  labs(title = "Rpact/East comparaison according to relevancy")
```




```
n_ratio_by_relevancy <-
  combined_res |>
  group_by(relevancy) |>
  summarize(
    min_n_ratio = min(n_ratio),
    q1_n_ratio = quantile(n_ratio, 0.25),
    mean_n_ratio = mean(n_ratio),
    med_n_ratio = median(n_ratio),
    q3_n_ratio = quantile(n_ratio, 0.75),
    max_n_ratio = max(n_ratio)
    )
```

```
e_ratio_by_relevancy <-
  combined_res |>
  group_by(relevancy) |>
  summarize(
    min_e_ratio = min(e_ratio),
    q1_e_ratio = quantile(e_ratio, 0.25),
    mean_e_ratio = mean(e_ratio),
    med_e_ratio = median(e_ratio),
    q3_e_ratio = quantile(e_ratio, 0.75),
    max_e_ratio = max(e_ratio)
    )
```

```
gt(n_ratio_by_relevancy)
```

relevancy	min_n_ratio	q1_n_ratio	mean_n_ratio	med_n_ratio	q3_n_ratio	max_n_ratio
low	0.9999966	1.0000013	1.0205471	1.0011145	1.0253553	1.077778
medium	0.7777778	0.9897901	0.9810277	0.9997638	1.0000000	1.000998
high	0.9864865	0.9917473	0.9952825	0.9968613	0.9989331	1.000000

```
gt(e_ratio_by_relevancy)
```

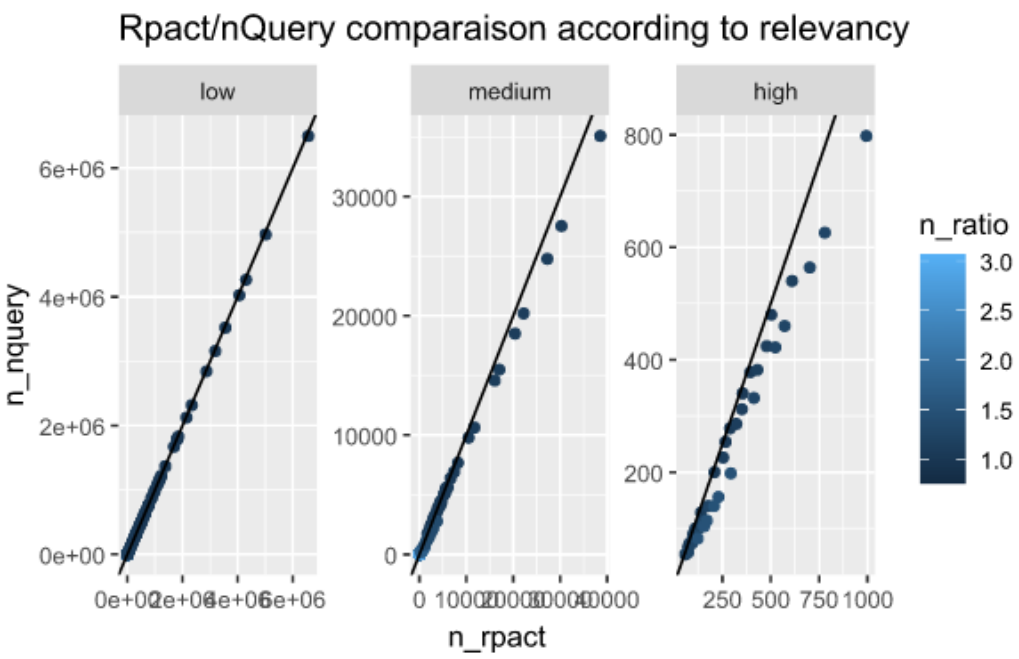
relevancy	min_e_ratio	q1_e_ratio	mean_e_ratio	med_e_ratio	q3_e_ratio	max_e_ratio
low	1	1.000007	1.023465	1.012611	1.036595	1.072464
medium	1	1.000000	1.000072	1.000000	1.000000	1.001232
high	1	1.000000	1.000000	1.000000	1.000000	1.000000

rpact vs nQuery

```
combined_res_2 <-
  p_res |>
  inner_join(
    nquery_res,
    by = join_by(alpha, power, hr,surv_3y),
    suffix = c("_rpact", "_nquery")
    ) |>
  mutate(
    e_ratio = e_rpact/e_nquery,
    n_ratio = n_rpact/n_nquery
  )
```

```
combined_res_2 <-
  combined_res_2 |>
    mutate(relevancy = evaluate_relevancy(alpha, power, hr, surv_3y)) |>
    mutate(relevancy = fct_relevel(relevancy,c("low", "medium", "high")))
```

```
ggplot(combined_res_2) +
  aes(x = n_rpact, y = n_nquery, color = n_ratio) +
  geom_point() +
  geom_abline() +
  facet_wrap(~relevancy, scales = "free") +
  labs(title = "Rpact/nQuery comparaison according to relevancy")
```




```
n_ratio_by_relevancy_2 <-
  combined_res_2 |>
  group_by(relevancy) |>
  summarize(
    min_n_ratio = min(n_ratio),
    q1_n_ratio = quantile(n_ratio, 0.25),
    mean_n_ratio = mean(n_ratio),
    med_n_ratio = median(n_ratio),
    q3_n_ratio = quantile(n_ratio, 0.75),
    max_n_ratio = max(n_ratio)
    )
```

```
e_ratio_by_relevancy_2 <-
  combined_res_2 |>
  group_by(relevancy) |>
  summarize(
    min_e_ratio = min(e_ratio),
    q1_e_ratio = quantile(e_ratio, 0.25),
    mean_e_ratio = mean(e_ratio),
    med_e_ratio = median(e_ratio),
    q3_e_ratio = quantile(e_ratio, 0.75),
    max_e_ratio = max(e_ratio)
    )
```

```
gt(n_ratio_by_relevancy_2)
```

relevancy	min_n_ratio	q1_n_ratio	mean_n_ratio	med_n_ratio	q3_n_ratio	max_n_ratio
low	0.750000	1.006643	1.256745	1.070473	1.348251	3.050000
medium	1.012485	1.070642	1.497513	1.341071	1.704502	3.071429
high	1.043165	1.094383	1.207266	1.180712	1.255098	1.490385

```
gt(e_ratio_by_relevancy_2)
```

relevancy	min_e_ratio	q1_e_ratio	mean_e_ratio	med_e_ratio	q3_e_ratio	max_e_ratio
low	1	1.000000	1.039892	1.000002	1.000566	2.000000
medium	1	1.000000	1.026703	1.000000	1.000616	1.333333
high	1	1.002044	1.007126	1.004436	1.012142	1.019608